

Theory Assignment II

Automata Theory Monsoon 2026, IIIT Hyderabad

September 11, 2026

Total Marks: 40 points

Due date: **17th September, 2026, 23:59:59**

General Instructions: All symbols have the usual meanings. All Turing Machines are assumed to be deterministic unless specified otherwise. AI usage and plagiarism is strictly **prohibited**. You are free to discuss the problems amongst yourselves, but your solution must be written in your own words.

1. [2 + 2 + 3 points] Show that the following languages are not context-free using the pumping lemma for CFLs, and construct deterministic total Turing Machines that decide them.

- $L_1 = \{a^{n!} : n \in \mathbb{N}\}$
- $L_2 = \{ww^Rw : w \in \{0,1\}^*\}$

[CO1, CO2, CO3, CO4]

2. [3 points] A robot on the infinite integer grid starts at the origin $(0,0)$ and reads a string over the alphabet $\{U, D, L, R\}$, each symbol moving it one unit up, down, left, or right respectively. A robot reads a move-string $w \in \{U, D, L, R\}^*$ and then reads a second string w' ; define

$L = \{w \# w' : w, w' \in \{U, D, L, R\}^*, w' \text{ retraces } w \text{ exactly (i.e. } w' = w) \text{ and } w \text{ returns to the origin}\}.$

Prove that L is not context-free. ($\#$ is a delimiter) [CO1, CO3]

3. [4+1+2 points]

a) Show that the following languages are undecidable via reductions to *HALT*:

- $L_1 = \{M : L(M) \text{ has no palindromes}\}$
- $L_2 = \{M : L(M) \text{ is regular}\}$

b) A property $\mathcal{P}$ of the recursively enumerable languages is a subset of the class of all RE languages; a Turing Machine M is said to satisfy $\mathcal{P}$ if $L(M) \in \mathcal{P}$. Let P be such that if for all TMs M_1, M_2 such that $L(M_1) = L(M_2)$ we have $M_1, M_2 \in P$. Then, a property $\mathcal{P}$ is *trivial* if it is satisfied by every TM or by no TM, and *nontrivial* otherwise.

Why is it that for every nontrivial property $\mathcal{P}$, the language $L_{\mathcal{P}} = \{\langle M \rangle : L(M) \in \mathcal{P}\}$ is undecidable? Explain intuitively in a few lines.

c) Justify why L_1 and L_2 are undecidable using b).

[CO1, CO3, CO4]

4. [2 points] Consider the language

$$L = \{\langle M, w \rangle : M \text{ halts on input } w \text{ within 100 steps}\}.$$

Determine whether L is decidable or undecidable. If it is undecidable, prove so by exhibiting a reduction from a known undecidable language. Otherwise, describe a Turing Machine that decides it and prove that this machine halts on every input.

[CO1, CO2, CO3, CO4]

5. [3 points] Show that for any language L over a finite alphabet Σ can be decided by a deterministic Turing Machine T with a single two-way infinite tape **if and only if** there exists a 2-stack PDA P that decides L .

Using this, deduce that k -stack PDAs are also equivalent to 2-stack PDAs, for fixed $k > 2$.

[CO1, CO2, CO4]

6. [1.5 + 1.5 + 2 + 1 + 3 points] Let Γ be an alphabet, and for a string $x \in \Gamma^n$, and a given Turing Machine T , define the *algorithmic complexity* C_T of the string x as:

$$C_T(x) := \min_p \{|p| : T(p) = x\}$$

In other words, it is the minimum length of an input p such that when T is run on p it produces x as its output. If no such p exists we define $C_T(x) = \infty$. Note that $C_T(x)$ may not be finite for every x , but for specific choices of T we can ensure it is:

a) Prove that if U is a universal Turing Machine, and T is an arbitrary Turing Machine, then:

$$C_U(x) \leq C_T(x) + b_T$$

where $b_T > 0$ is a constant dependent only on T .

b) Deduce that:

$$C_U(x) \leq n + O(1)$$

We can interpret algorithmic complexity differently: Consider inputs p to be *programs*, which when run (i.e simulated on a universal Turing Machine) output a string x . Algorithmic complexity hence asks what the shortest program is, that when run, outputs x . We can show that this is not dependent on the choice of U (or programming language).

c) Show that for any two universal Turing Machines U, V :

$$|C_U(x) - C_V(x)| \leq c$$

where $c > 0$ is a constant depending only on U, V .

For example, if U was a universal Turing Machine that ran Python programs, and V similarly ran C programs, for large enough strings there is only a constant factor additive difference in their algorithmic complexity. Given this invariance, we will drop the subscript and refer to the algorithmic complexity as $C(x)$. Assume $\Gamma = \{0, 1\}$ from now on.

Algorithmic complexity can be thought of as a measure of how "random" a string is, or equivalently how "compressible" a string is. Highly compressible strings, despite having massive length, can have short programs, and hence small algorithmic complexity. However, there exist an exponentially growing number of incompressible strings as well:

- d) Let b_k be the binary form of $f(k) = 2^{f(k-1)}$, $k > 1$, such that $f(1) = 2$. Show that

$$C(b_k) \leq \log k + O(1)$$

- e) For $c > 0$, we call a string c -incompressible if $C(x) \geq |x| - c$. Prove that there are at least $2^n - 2^{n-c} + 1$ strings of length n that are c -incompressible.

[CO1, CO2, CO3, CO4]

7. [2 + 2 + 3 + 2 points] Let U be a universal Turing Machine, and consider the following probability measure $P(x)$, which tries to assign the probability that, for a fixed string $x \in \{0, 1\}^*$, a randomly chosen input $p \in \{0, 1\}^*$ when run on U outputs x :

$$x \mapsto P(x) = \sum_{p: U(p)=x} 2^{-|p|}$$

- a) Prove that $P(x)$ diverges to infinity for all x , and thus such a probability measure P cannot exist for the standard Turing Machine model.

To tackle this issue, we introduce the *self-delimiting Turing Machine* T , which is such that it has three infinite tapes:

- A read-only input tape containing the input p which is *one way* i.e the tape head only moves towards the right.
- A write-only output tape which is initially blank and is also one way.
- A work tape which can read and write and is initially blank.

We say a *successful* computation is when T halts with the tape head of the input tape on the *last* bit of the input p and no further; if in this case the output tape contains the string x , we say $T(p) = x$. In any other case we say that the computation *failed* and say the output $T(p)$ is undefined. It is not hard to show that there exists a universal self-delimited Turing Machine U (you can assume this).

We say a non-empty set of strings $S \subseteq \{0, 1\}^*$ is *prefix-free* if no string appears as the prefix of another; i.e for all $p, q \in S$, $s \in \{0, 1\}^*$ such that $p \neq q$, we have that $p \neq qs$.

- b) Prove that given a self-delimited Turing Machine T , the set of inputs p for which it halts is prefix-free. Show then that for any x the set $\{p \in \{0, 1\}^* : U(p) = x\}$ is prefix-free.
- c) Prove Kraft's Inequality; for any prefix-free set $S \subseteq \{0, 1\}^*$, we have

$$\sum_{x \in S} 2^{-|x|} \leq 1$$

(Hint: Consider an infinite binary sequence $B = b_1b_2 \dots$ such that every bit is independently chosen as 0 with probability $\frac{1}{2}$, else 1, and consider $\Pr[\bigcup_{x \in S} E_x]$, where E_x is the event that x is a prefix of B .)

- d) Deduce that $P(x)$ defined on the self-delimiting Turing Machine model can now be a valid probability measure over $\{0, 1\}^*$ by showing that $P(x) \leq 1$ and $\sum_x P(x) \leq 1$.

[CO1, CO2, CO3]

END OF ASSIGNMENT
